Sokrio-তে Kafka — সিস্টেম ডিজাইন ও DevOps গাইড (বাংলা)

> `sokrio-server`-এ নতুন জয়েন করা ডেভদের জন্য লেখা। এটা Kafka-র জেনারেল থিওরির বই না — আমরা কীভাবে এটা ইউজ করছি, কেন এভাবে ডিজাইন করা হয়েছে, আর কোন কোন প্রোডাকশন ইনসিডেন্টের পর এই শেপ-এ এসেছে সেই গল্প। 2026-09-08-এ লেখা, পুরোটাই `sokrio-server`-এর `git log` ঘেঁটে বানানো (GitHub PR না, কারণ এই environment-এ `gh` CLI নেই আর API token-ও নেই — নিচের প্রতিটা hash `git show <hash>` দিয়ে নিজে চেক করে নিতে পারেন)।

---

১. আমাদের কোডবেসে Kafka কেন লাগলো

Sokrio একটা মাল্টি-টেন্যান্ট Laravel মনোলিথ। বেশিরভাগ কাজ সিঙ্ক্রোনাসলি হয় — হয় একটা HTTP রিকোয়েস্টের মধ্যেই, নয়তো cron দিয়ে চলা ব্যাচ জবে (`app/Console/Kernel.php` খুললেই দেখবেন `dailyAt()`/`hourlyAt()`-এ ভরা, সব queue job dispatch করছে)। সাধারণ CRUD-এর জন্য এই approach একদম ফাইন, কিন্তু দুই জায়গায় এসে আটকে যায়:

1. **অনেক বেশি ফ্রিকোয়েন্সিতে আসা, প্রতিটার মূল্য কম এমন ডেটা** — যেমন ফিল্ড রেপ-দের GPS ট্র্যাকিং পিং। প্রতিটা পয়েন্ট রিকোয়েস্টের মধ্যেই সিঙ্ক্রোনাসলি লিখতে গেলে অযথা মোবাইল অ্যাপ স্লো হয়ে যাবে, অথচ ইউজার সেটার কোনো বেনিফিট-ই পাচ্ছে না।
2. **যেটা প্রায়-রিয়েলটাইম হওয়া উচিত ছিল, কিন্তু আসলে ঘণ্টা/রাতের ব্যাচ জব চলার পরেই ঠিক হয়** — যেমন অর্ডার/ডেলিভারি সামারি এখন ঘণ্টায় একবার আপডেট হয়। এই ফোল্ডারেরই `primary-order-delivery-kafka-plan.md`-তে এর একটা বাস্তব কেস আছে, পড়ে দেখতে পারেন।

Kafka মূলত একটা টেকসই, অর্ডারড বাফার — "কিছু একটা ঘটলো" (producer, বেশিরভাগ সময় একটা রিকোয়েস্টের ভেতরে) আর "যাও, ভারী কাজটা করে ফেলো" (consumer, নিজের মতো আলাদা প্রসেসে চলে, রিকোয়েস্ট লেটেন্সি বা cron শিডিউল কোনোটার সাথেই বাঁধা না) — এই দুইয়ের মাঝে।

এটা কিন্তু "সবকিছুর জন্য pub/sub" না। আজকের দিনে মাত্র একটা টপিকেরই সত্যিকারের লাইভ consumer আছে — `tracking` (§৩-এ বিস্তারিত)। আরও দুইটা (`order_summaries`, `delivery_summaries`) কোডে বসানো আছে, কিন্তু তাদের publish কল এখনও কমেন্ট-আউট করা। তাই `TopicRegistry`-তে থাকা মানেই লাইভ চলছে — এটা ধরে নেবেন না, `ConsumeCommand::HANDLERS` দেখে নিশ্চিত হন।

---

২. মেন্টাল মডেল — Kafka-র বেসিক জিনিসগুলো, আমাদের ভাষায়

জিনিস	আমাদের এখানে যার মানে
Topic	একটা নাম-দেওয়া স্ট্রিম, যেমন `tracking.events.v1`। প্রতি বিজনেস ইভেন্টের জন্য একটা করে। `TopicRegistry::TOPICS`-এ ডিফাইন করা আছে, আর production ছাড়া বাকি environment-এ নামের পেছনে অটোমেটিক suffix লাগে (`tracking.events.v1.staging`) — যাতে একই broker শেয়ার করলেও environment-গুলো একটা আরেকটার সাথে মিশে না যায়।
Partition	একটা topic-কে N ভাগে ভাগ করা হয় (ডিফল্ট `KAFKA_AUTO_CREATE_PARTITIONS=3`), যাতে parallel-এ কাজ করা যায়। Kafka শুধু একটা partition-এর ভেতরে অর্ডার গ্যারান্টি দেয়, পুরো topic জুড়ে না।

জিনিস	আমাদের এখানে যার মানে
Partition key	আমরা publish করি `key = tenant_id` দিয়ে (`KafkaProducer::publish()`, ডিফল্ট `partitionKey ?? (string)$tenantId`)। একই key মানে একই partition — মানে **একটা টেন্যান্টের সব ইভেন্ট সবসময় ঠিক ওই অর্ডারেই আসবে**। কোনো লক ছাড়াই টেন্যান্ট-অনুযায়ী অর্ডারিং পাওয়ার আসল ট্রিকটাই এটা।
Producer	যে কেউ একটা ইভেন্ট লেখে — সাধারণত রিকোয়েস্ট-টাইম Controller বা model Observer। সিঙ্ক্রোনাস, timeout খুবই ছোট (1s), broker ডাউন থাকলেও কখনো রেসপন্স আটকে রাখবে না (§৫)।
Consumer	দীর্ঘসময় ধরে চলতে থাকা `php artisan kafka:consume {topic}` প্রসেস (প্রতি topic-এ একটা), supervised, কিন্তু queue worker না — Redis/DB queue টেবিলে নির্ভর করে না, broker-এর নিজের offset ট্র্যাকিং-এর উপর নির্ভর করে।
Consumer group	একই `group_id`-তে (`KAFKA_GROUP`, ডিফল্ট `sokrio-group`) একই topic পড়া সব consumer একে অপরের সাথে partition ভাগ করে নেয়, আর কেউ join/leave করলে *rebalance* হয়। আমরা এখন প্রতি topic-এ `numprocs=1` রাখি (§৭), তাই rebalance মূলত রিস্টার্টের সময়ই ঘটে, স্কেল-আউটের জন্য না — তবে group membership-এর প্লাম্বিংটা (§৪.৫) ঠিকঠাক বসাতেই হয়েছিল।
Offset	প্রতি-পার্টিশনের read কার্সার — মানে "কতদূর পড়া হয়েছে"। Offset commit মানে "এই পর্যন্ত সব ডান হয়ে গেছে"; consumer রিস্টার্ট হলে শূন্য থেকে না, শেষ committed offset থেকেই আবার শুরু করে।
Delivery semantics	একটা flush শেষে handler রান হওয়ার পরই আমরা offset commit করি, কিন্তু handler-টা এমনভাবে লেখা যে সে **কখনো throw করে না** — কোনো টেন্যান্ট 3 বার ফেল করলেও সেটা শুধু লগ হয়, ব্যাচ তারপরও commit হয়ে যায়। মানে ফেল হওয়া অংশের জন্য এটা আসলে **at-most-once**, at-least-once না — যদিও offset manually কন্ট্রোল করছি। এটা ইচ্ছাকৃত ট্রেড-অফ (§৪.৪), নতুন কোনো handler লেখার আগে জেনে রাখা ভালো — "Kafka-তে গেছে মানেই একদিন না একদিন ল্যান্ড করবে" — এই ধারণা এখানে খাটে না।

---

৩. আসল জিনিসগুলো, ফাইল ধরে ধরে

```
app/Contracts/Kafka/
  MessageProducerInterface.php     — publish()/publishBatch() কনট্র্যাক্ট
  MessageConsumerInterface.php     — consume($topic, callable $handler,
$maxRuntime)
  KafkaMessageHandlerInterface.php — handle() [deprecated] + handleBatch()
  TopicRegistryInterface.php
  ConsumerFactoryInterface.php     — টেস্টের জন্য Consumer construction-এর
seam
  TopicProvisionerInterface.php    — মিসিং topic থাকলে অটো বানিয়ে দেয়

app/Services/Kafka/
  KafkaProducer.php        — write সাইডে broker-এর সাথে কথা বলা একমাত্র ক্লাস
  KafkaConsumer.php        — read সাইডে broker-এর সাথে কথা বলা একমাত্র ক্লাস
  TopicRegistry.php        — topic-key → topic-name + event-type কনস্ট্যান্ট
  TopicProvisioner.php     — প্রথমবার ইউজ হলে topic বানিয়ে দেয় (dev/staging-এর
সুবিধা)
  Signal/ShutdownFlag.php  — SIGTERM/SIGINT এলে সেট হওয়া একটা প্রসেস-ওয়াইড
ফ্ল্যাগ
  Socket/InterruptResilientStreamSocket.php — EINTR-safe socket (§৪.২)
```

```
  Handlers/
    TrackingMessageHandler.php     — একমাত্র লাইভ, প্রমাণিত হওয়া handler

app/Console/Commands/Kafka/ConsumeCommand.php  — `kafka:consume {topic}`,
HANDLERS ম্যাপ
app/Observers/Concerns/PublishesKafkaMessage.php — Observer-দের শেয়ার্ড
try/catch/log/publish trait
app/Observers/{Order,Delivery,DepartmentChange}Observer.php — publish কল
সাইট (Order/Delivery: কমেন্ট-আউট)
app/Providers/AppServiceProvider.php  — observer রেজিস্ট্রেশন
config/kafka.php                       — যাবতীয় টিউনেবল, §৭-এ ডিটেইল
```

একমাত্র লাইভ পাইলট (Tracking)-এর ডেটা কোন পথে যায়:

```
মোবাইল অ্যাপ
   │  POST /trackings  (একটা) বা /v2/trackings (বাল্ক)
   ▼
TrackingController::store()
   │  ১. MySQL (Tracking) + Mongo (TrackingReport)-এ যথারীতি সিঙ্ক্রোনাসলি লেখে
   │  ২. এর সাথে MessageProducerInterface::publish()/publishBatch()-ও কল
করে
   ▼                                                  KafkaProducer
   │  একটা envelope বানায় {event_id, event_type, tenant_id, occurred_at,
payload}
   │  key = tenant_id → একই টেন্যান্টের ইভেন্ট একই partition-এ পড়ে
   ▼
Kafka topic: tracking.events.v1[.env]
   ▲
   │  Supervisor-এ চলতে থাকা লম্বা প্রসেস
kafka:consume tracking  (ConsumeCommand)
   │  HANDLERS ম্যাপ থেকে TrackingMessageHandler খুঁজে বের করে
   ▼
KafkaConsumer::consume()
   │  নিজের poll loop নিজে চালায় (লাইব্রেরির Consumer::start() ইউজ করে না)
   │  batch_size (200) বা batch_interval (1.0s) — যেটা আগে হয়, ততক্ষণ মেসেজ
জমায়
   │  পুরো ব্যাচ একবারে handler->handleBatch($buffer)-কে দিয়ে দেয়
   │  প্রতি flush-এ প্রতি-partition একবার করে offset commit করে, মেসেজ ধরে ধরে না
   ▼
TrackingMessageHandler::handleBatch()
   │  tenant_id দিয়ে গ্রুপ করে
   │  প্রতিটা টেন্যান্টের জন্য: TenantManager::tenant($org)->reconnect(), তারপর
   │  পুরো স্লাইসের জন্য একবারেই WriteBulkTrackingActions() কল
   │  ফেইল করলে 3 বার রিট্রাই, তারপরও না হলে এক-এক পয়েন্ট করে লেখায় নেমে আসে
   │  handleBatch থেকে কখনো throw করে বের হয় না — লগ করে এগিয়ে যায়
   ▼
Tracking (MySQL) + TrackingReport (Mongo) — সিঙ্ক পাথ যেই টেবিলে লেখে, ওই একই
টেবিল
```

লক্ষ্য করার মতো বিষয় — ধাপ ১-এর সিঙ্ক্রোনাস write কিন্তু **কখনোই সরানো হয়নি**। মানে আজকের দিনে Tracking-এর জন্য Kafka একটা *প্যারালাল/শ্যাডো সিগন্যাল*, সোর্স অফ ট্রুথ না। consumer আর sync path — দুটোই একই টেবিলে লিখছে, তাই থিওরিটিক্যালি ডুপ্লিকেট-write হওয়ার চান্স আছে। এটা পুরোপুরি cut-over হয়নি এখনও — টিমের কাছে বর্তমান স্ট্যাটাসটা একবার কনফার্ম করে নেওয়া ভালো, ধরে না নিয়ে।

---

৪. রিলায়েবিলিটি — আসল ইনসিডেন্টগুলোর গল্প

এই ডকের সবচেয়ে দামি অংশ এটাই। প্রোডাকশনে বিনা সুপারভিশনে ভরসা করে চালানোর মতো বিশ্বাসযোগ্য হওয়ার আগে Kafka এখানে **দুইবার পুরোপুরি revert** হয়েছে, আর তারপর লম্বা একটা হার্ডেনিং কমিটের লাইন গেছে। এগুলো ক্রমানুসারে পড়লে যেকোনো টিউটোরিয়ালের চেয়ে বেশি শিখবেন PHP-তে Kafka চালানো নিয়ে।

৪.১ — শুরুর দিকের ব্যর্থ চেষ্টাগুলো (2026-এর প্রথম দিকে)

```
dcc8984e8  add-kafka                                — প্রথম experiment
c01fe1905  partition-added-in-kafka                 — সাথে সাথেই revert
(0598abcd6)
22e866c83  kafka-implementation-for-order-summary   — PR #4044 হিসেবে merge
হয়েছিল...
ad4aa8aa7  Revert "Kafka implementation for order summary master"  —
...তারপর revert (PR #5112)
d85dcb4bc  Add Kafka producer/consumer and tracking pipeline       — PR
#5113 হিসেবে merge...
2b0587fd0  Revert "Add Kafka producer/consumer and tracking pipeline"  —
...revert (#5115)...
                                                                    —
...শেষে আবার merge (#5116)
```

যা শেখার আছে: বর্তমান ডিজাইনটা টিকে যাওয়ার আগে দুইটা আলাদা "Kafka লাগাই" চেষ্টা (আগে order-summary, তারপর একদম নতুন করে tracking pipeline) merge হয়ে তারপর revert হয়েছে। নতুন কোনো Kafka consumer-এর প্রথম ভার্সন ডিজাইন করার সময় ধরে নিন — প্রথমবারেই সব ঠিকঠাক হবে না, কিছু failure mode মিস হবেই। তাই শুরু থেকেই একটা সস্তা rollback পথ রাখুন — publish কল feature-flag দিয়ে বন্ধ করার সুযোগ, বা এমন একটা consumer যেটা বন্ধ করলেও ডেটা হারাবে না কারণ sync path তখনও চলছে। ঠিক যেভাবে `d85dcb4bc` নতুন Kafka publish-এর পাশাপাশি Tracking-এর পুরনো সিঙ্ক write-টাও রেখে দিয়েছিল।

৪.২ — শ্যাডো-রাইট দিয়ে যাচাই, তারপর সেটা সরানো

`d85dcb4bc` (যে tracking pipeline-টা শেষমেশ টিকে গেছে) শুরুতে consume করা মেসেজ সরাসরি আসল `TrackingReport`-এ না লিখে একটা **অস্থায়ী শ্যাডো Mongo কালেকশনে** (`TrackingNew`/`tracking_reports_new`) লিখতো — যাতে Kafka থেকে আসা ভ্যালু আর পুরনো সিঙ্ক পাথের ভ্যালু পাশাপাশি রেখে মিলিয়ে দেখা যায়, নতুন পাইপলাইনটা বিশ্বাস করার আগে।

```
d32d99105  Remove TrackingNew and re-enable consumer writes

  শ্যাডো-রাইট যাচাইয়ের পর্ব শেষ: অস্থায়ী TrackingNew মডেল ডিলিট করা হলো এবং
  Kafka tracking consumer আবার Tracking (MySQL) ও TrackingReport (Mongo)-এ
```

```
  সরাসরি লেখা শুরু করলো। TrackingController-ও আপডেট হলো যাতে Kafka ইভেন্ট
  publish করে ও validated payload সহ HTTP 202 রিটার্ন করে।
```

যা শেখার আছে: নতুন কোনো async write path-কে এখনও পুরোপুরি বিশ্বাস না করলে, আসলটার একদম একই shape-এর একটা শ্যাডো কালেকশন দিন, কিছুদিন প্যারালালে চালিয়ে দুটো মিলিয়ে দেখুন — মিললে তবেই শ্যাডো মডেল সরিয়ে আসল কালেকশনে cut-over করুন। অপ্রমাণিত একটা পাইপলাইন থেকে সরাসরি আসল টেবিলে লেখা শুরু করবেন না। এই জন্যই Primary Order/Delivery-র প্ল্যানটা নতুন করে field-mapping লজিক না বানিয়ে বিদ্যমান, ইতিমধ্যে প্রমাণিত `SummariesQuery`-র লজিকটাই পুনর্ব্যবহার করছে — দুটো আলাদা mapping থাকলে সেগুলো চুপচাপ একটা থেকে আরেকটা ড্রিফট করে যেতে পারতো।

৪.৩ — EINTR: pure-PHP ক্লায়েন্ট signal-interrupted syscall সামলাতে পারে না

```
fb2b932eb  Make Kafka resilient to signal-interrupted syscalls; isolate its
logs

  pure-PHP longlang/phpkafka ক্লায়েন্টের
stream_select()/stream_socket_client()-এ
  কোনো EINTR হ্যান্ডলিং নেই। প্রোডাকশনে Datadog profiler-এর টাইমার সিগন্যাল এগুলোকে
  বারবার interrupt করে দিচ্ছিলো, যার ফলে `Kafka publish failed` (FPM পাথে
  ErrorException) আর consumer-এ `Could not read N bytes from stream (not
readable)`
  — এমনকি kafka:consume কমান্ডও মাঝ-poll-এ ক্র্যাশ করে যাচ্ছিলো।
```

আসল কারণ: Datadog-এর **continuous profiler** কল স্ট্যাক স্যাম্পল করার জন্য প্রায় ~100 Hz-এ টাইমার সিগন্যাল পাঠায়। pure-PHP Kafka ক্লায়েন্টের যেকোনো blocking syscall (`stream_select`, `stream_socket_client`) এই সিগন্যালের কোনোটার হাতে পড়ে interrupt (`EINTR`) হয়ে যেতে পারে, আর PHP-র socket লেয়ার C ক্লায়েন্টের `SA_RESTART`-এর মতো নিজে থেকে রিট্রাই করে না।

সমাধান: `InterruptResilientStreamSocket` — একটা deadline-এর ভেতরে থেকে EINTR-এ থেমে যাওয়া select()/connect() আবার রিট্রাই করে, মিথ্যা error propagate না করিয়ে। producer ও consumer দুটোতেই `ProducerConfig`/`ConsumerConfig::setSocket()`-এর মাধ্যমে বসানো আছে।

অপারেশনালি এর প্রভাব: `DD_PROFILING_ENABLED=false` আর `DD_TRACE_CLI_ENABLED=false` **শুধু** Kafka consumer-এর Supervisor প্রোগ্রামে সেট করা (`environment=` লাইনে, পুরো container-এ না — §৭)। এটা ভুলে রয়ে যাওয়া কোনো workaround না, ইচ্ছাকৃত সিদ্ধান্ত — যে প্রসেস এমনিতেই সারাক্ষণ চলে, তার জন্য profiler শুধু বাড়তি retry churn তৈরি করে, কোনো লাভ দেয় না।

যা শেখার আছে: লোডের সময় প্রোডাকশনে একটা random, transient socket ফেইলিওরের মতো লাগছে এমন কোনো Kafka error ডিবাগ করছেন, কিন্তু লোকালি কিছুতেই reproduce হচ্ছে না — তাহলে নেটওয়ার্ক ইস্যু ধরে নেওয়ার আগে একবার APM/profiling সিগন্যাল এখানে ভূমিকা রাখছে কিনা চেক করে নিন।

৪.৪ — Static বনাম dynamic consumer group membership (KIP-345)

```
de9b53f38  Disable static group membership and handle Kafka errors
d1cbf7c54  Enable dynamic Kafka group ID and retry errors
```

দুইটা কমিট, দেখতে দুইটা উলটো দিকে যাচ্ছে মনে হয় — কিন্তু কেন, সেটা বোঝা দরকার। Kafka-র KIP-345 একটা consumer-কে একটা *static* group-instance-id রাখার সুযোগ দেয়, যাতে ছোট একটা রিস্টার্টে পুরো group-এর rebalance না ঘটে। আমরা `groupInstanceId`-কে `null` করে এটা বন্ধ করতে গিয়েছিলাম — কিন্তু তাতে join-group হ্যান্ডশেকই ক্র্যাশ করে যাচ্ছিলো, কারণ এই ভার্সনের ক্লায়েন্টে `GroupManager::syncGroup()`/`heartbeat()`-এর `$groupInstanceId` প্যারামিটারটা **non-nullable**। সমাধানটা "static membership আবার চালু করো" ছিল না — বরং "`null` না দিয়ে explicitly `''` (খালি string) দাও", যেটাকে লাইব্রেরি *dynamic* (non-static) membership হিসেবে ধরে, আর type error-ও হয় না।

দুইটা কমিটেই `longlang\phpkafka\Exception\KafkaErrorException`-এর error হ্যান্ডলিং আরও বাড়ানো হয়েছে — broker-সাইড হ্যান্ডশেক ফেইলিওর যেমন `FENCED_INSTANCE_ID` (82), যেটা কোনো পুরনো/ডুপ্লিকেট group member থেকে গিয়ে ঘটে, সেগুলোকে এখন transient ধরে নিয়ে পুরো consumer ক্র্যাশ না করিয়ে join/reconnect রিট্রাই করা হয়।

যা শেখার আছে: কোনো লাইব্রেরির ডিফল্ট দেখতে "সঠিক" সেটিং মনে হলেও (যেমন `null` মানে "off") সেটা Kafka প্রোটোকল স্পেক অনুযায়ী ঠিক তেমনই কাজ করবে ধরে নেবেন না — ক্লায়েন্টের আসল type signature-টা চেক করে দেখুন। আরেকটা কথা — broker-সাইড group-coordination error একটা দীর্ঘস্থায়ী consumer-এর জন্য স্বাভাবিক ও প্রত্যাশিত একটা ব্যাপার, ক্র্যাশ করানোর মতো কিছু না। কোনো Kafka exception পেলে আগে জিজ্ঞেস করুন — "broker কি শুধু আমাকে হ্যান্ডশেকটা আবার করতে বলছে?"

৪.৫ — মাঝপথে সিগন্যাল পড়ায় DB কানেকশন করাপ্ট হয়ে যাওয়া

```
2df7e23b0  Harden Kafka consumer against DB reconnect issues

  এই চেঞ্জ প্রতিটা Kafka মেসেজ প্রসেস করার সময় SIGTERM/SIGINT ব্লক করে দেয়, যাতে
  shutdown সিগন্যাল একটা চলমান DB read-কে মাঝপথে থামিয়ে landlord কানেকশনের
  স্টেট নষ্ট করে দিতে না পারে। কোনো টেন্যান্ট লুকআপে স্টেল/করাপ্টেড landlord
  কানেকশন পেলে, tracking handler এখন রিট্রাইয়ের আগে DB purge ও reconnect করে,
  আর রিকভারিটা লগ করে রাখে।
```

এই জন্যই `KafkaConsumer::flush()`-এ (বর্তমান কোডে) handler কলটা `pcntl_sigprocmask(SIG_BLOCK, [SIGTERM, SIGINT])` / `SIG_UNBLOCK`-এর মধ্যে মোড়ানো — কারণ একটা DB read মাঝপথে interrupt হলে wire-এ কিছু বাড়তি বাইট থেকে যায়, যেটা কানেকশনের protocol state নষ্ট করে দেয় এবং *পরের* কোনো একটা অসম্পর্কিত কোয়েরিতে গিয়ে আজব একটা error দেয় ("Packets out of order")। `TrackingMessageHandler::resolveOrg()`-ও নিজেকে সারিয়ে নিতে পারে — landlord কানেকশনে সমস্যা পেলে `DB::purge()` + `DB::reconnect()` করে একবার রিট্রাই করে, error না ছুঁড়ে।

যা শেখার আছে: PHP-তে signal-safety মানে শুধু "loop থামাতে SIGTERM ধরা" না। কোনো সিগন্যাল যদি একটা blocking DB কলের ঠিক মাঝখানে পড়ে যায়, সেটা connection state-এ এমন গণ্ডগোল করে দিতে পারে যা শুধু পরের কোনো একদম অন্য কোয়েরিতে গিয়ে বোঝা যায়। তাই ইন্টারাপ্ট-সেফ না এমন কোনো critical section-এর চারপাশে সিগন্যাল ব্লক করে রাখুন।

৪.৬ — Redis/cache ফেইল করলে সেটা Kafka-র সমস্যা বানাবেন না

```
37840a3b5  Harden feature tracking, flush & Kafka handling

  ... Kafka পাথ হার্ডেন করা হলো: Redis/Predis cache ফেইলিওরকে ট্রান্সপোর্ট নয়েজ
```

```
হিসেবে ধরা, ডিফল্টভাবে Sentry-তে অযথা রিপোর্ট না পাঠানো (opt-in কনফিগ যোগ করে),
আর TopicProvisioner-কে cache error-এর সামনে টিকে থাকার মতো বানানো।
```

`TopicProvisioner` cache ব্যবহার করে যাতে প্রতিটা publish-এ বারবার "এই topic-টা আছে তো?" চেক না করতে হয়। Redis-এ একটু ঝামেলা হলেই যদি পুরো Kafka publishing বন্ধ হয়ে যায়, সেটা ভুল — তাই এই ফিক্সে cache ফেইলিওরকে cache miss-এর মতোই ধরা হয়েছে, মানে সরাসরি broker-এ গিয়ে চেক করে নেয়, Redis-এর error আটকে না গিয়ে।

যা শেখার আছে: একটা সেকেন্ডারি ডিপেন্ডেন্সি (cache) ফেইল করলে সেটা মূল পাথের স্পিড কমাতে পারে, কিন্তু সঠিকতা বা availability নষ্ট করা উচিত না। এই একই প্যাটার্ন `Org::findById()`-তেও (`app/Models/Core/Org.php`) আছে — Redis দিয়ে cache করা, কিন্তু `ConnectionException`/`RedisException` হলে সরাসরি DB-তে fallback করে।

৪.৭ — এক-মেসেজ থেকে ব্যাচে যাওয়া (থ্রুপুট রিরাইট)

```
a7e63f4bb  Kafka consumer: batch processing & graceful shutdown
e99d02a03  Add bulk tracking endpoint and Kafka batch support
```

আগে consumer একবারে একটা মেসেজ প্রসেস করতো — প্রতিটা মেসেজের জন্য আলাদা DB write, আলাদা offset commit। `a7e63f4bb`-তে এটা পুরো রিরাইট হয় — এখন ডিকোড করা মেসেজ বাফারে জমা থাকে আর `batch_size=200` বা `batch_interval=1.0s` — যেটা আগে হয়, তখন flush হয়। handler কনট্র্যাক্টে `handleBatch()` যোগ হলো (আর পুরনো `handle()` শুধু `@deprecated` single-message shim হিসেবে থেকে গেলো), আর offset commit-ও এখন **প্রতি partition প্রতি flush**-এ একবার হয়, প্রতি মেসেজে না। ঠিক কীভাবে এটা কাজ করে, বর্তমান কোডে `KafkaConsumer::pollLoop()`/`flush()` দেখলেই বুঝবেন (ফাইল ম্যাপ §৩-এ আছে)।

যা শেখার আছে: নতুন কোনো consumer handler লেখার সময় `handleBatch()`-টাই আসল কনট্র্যাক্ট — আগে `tenant_id` দিয়ে গ্রুপ করুন (প্রতি টেন্যান্ট প্রতি flush-এ একবার DB reconnect + একবার bulk write), মেসেজ একটা একটা করে প্রসেস করবেন না — যদিও interface টেকনিক্যালি এখনও একটা single-message `handle()` মেথড রাখতে দেয়।

৪.৮ — পিরিয়ডিক রিসাইকেল টিউন করা

```
6486a0607  Increase default Kafka consumer max runtime to 6h
```

Consumer প্রতি `KAFKA_CONSUMER_MAX_RUNTIME` সেকেন্ড পর নিজে থেকেই পরিষ্কারভাবে exit করে যায়, যাতে Supervisor একে ফ্রেশ করে রিস্টার্ট করে (দীর্ঘসময় চলা একটা PHP প্রসেসে মেমরি বেড়ে যাওয়াটা এভাবে বাউন্ড করা হয়)। শুরুতে এটা কনজারভেটিভভাবে 1h ছিল, পরে 6h-এ বাড়ানো হয়েছে — কারণ দেখা গেলো প্রতিবার রিসাইকেলে একটা পুরো consumer-group rebalance হয়, আর এত ঘনঘন সেই খরচটা পোষায় না।

যা শেখার আছে: "পিরিয়ডিক রিসাইকেল" কোনো ক্র্যাশ না। `kafka.log`-এ প্রায় 6h পরপর `Kafka consumer: clean shutdown` দেখলে ভয় পাবেন না — এটা ঠিক যেভাবে ডিজাইন করা, সেভাবেই কাজ করছে।

---

৫. এই সবকিছু থেকে বের হওয়া ডিজাইন প্রিন্সিপল

1. **Kafka ফেইল করলে যেন রিকোয়েস্ট না ভাঙে।** Producer-এর timeout ছোট রাখা (1s connect/send/recv — `config/kafka.php`-র `producer` অংশে); `KafkaProducer::publish()` exception ধরে ফেলে, লগ

করে, আর `null` রিটার্ন করে। Caller-রা `null` পেলে সেটাকে "ইভেন্ট publish হয়নি" হিসেবে ধরে, error হিসেবে না — দেখুন `PublishesKafkaMessage::publishKafkaMessage()`, যেটা নিজেও সবকিছু try/catch-এ মুড়ে রাখে।

2. **একটা handler ফেইল করলে যেন পুরো consumer না থামে।** `handleBatch()`-এর বাইরে কখনো throw করা যাবে না — ধরুন, `kafka` চ্যানেলে লগ করুন, সত্যিই অপ্রত্যাশিত কিছু হলে `report()` করুন, তারপর এগিয়ে যান। একটা খারাপ টেন্যান্ট/মেসেজের কারণে পুরো ব্যাচের offset commit আটকে থাকবে না।
3. **Consumer-এর লগ Sentry/Slack থেকে ইচ্ছাকৃতভাবে আলাদা রাখা হয়েছে** (`kafka` লগ চ্যানেল, `stack` না) — কারণ রিট্রাই, রিকানেক্ট, পিরিয়ডিক clean-shutdown রিসাইকেল — এসব স্বাভাবিক ব্যাকগ্রাউন্ড নয়েজ, ইনসিডেন্ট না। সত্যিই অপ্রত্যাশিত error হলেই শুধু `report()` হয়।
4. **Partition key = tenant_id, সবসময়** — নির্দিষ্ট কোনো কারণ না থাকলে (`publish()`-এর `$partitionKey` ডিফল্ট `(string) $tenantId`)। কোনো লক ছাড়াই প্রতি-টেন্যান্ট অর্ডারিং পাওয়ার এটাই মূল রহস্য।
5. **Handler-এর ভেতরে tenant দিয়ে গ্রুপ করুন, প্রতি টেন্যান্ট প্রতি flush-এ একটা DB reconnect** — মেসেজ ধরে ধরে না। `TenantManager::tenant($org)->reconnect()` ফ্রি না, তাই পুরো ব্যাচের স্লাইস জুড়ে এর খরচটা ভাগ করে নিন।
6. **একটা ব্যাচে বারবার আসা read লুকআপ cache করে রাখুন** (`TrackingMessageHandler::$orgs`-এ `Org::findById`-এর মতো) — যেকোনো real ব্যাচে গুটিকয়েক টেন্যান্টই বারবার ঘুরেফিরে আসে।
7. **Critical DB write সেকশনের চারপাশে সিগন্যাল ব্লক করুন** (`pcntl_sigprocmask`) — শুধু poll loop-এর চারপাশে না, ভেতরেও।
8. **নতুন handler মানেই — `TopicRegistry`-তে নতুন topic key, `ConsumeCommand::HANDLERS`-এ নতুন এন্ট্রি, নতুন Supervisor প্রোগ্রাম।** এখানে কোনোকিছু স্বয়ংক্রিয়ভাবে discover হয় না, ইচ্ছাকৃতভাবেই একটা explicit dispatch table রাখা হয়েছে যাতে রিভিউ করা যায় (দেখুন reflection guard টেস্ট, `tests/Unit/Console/Kafka/ConsumeCommandHandlersTest.php`, যেটা নিশ্চিত করে প্রতিটা রেজিস্টার্ড handler আসলেই interface ইমপ্লিমেন্ট করছে)।

---

৬. মাল্টি-টেন্যান্ট মোড় — এটা টেক্সটবুক Kafka না

প্রতিটা envelope-এ `tenant_id` থাকে (`KafkaProducer::buildEnvelope()`):

```
{
  "event_id": "uuid-v4",
  "event_type": "tracking.created",
  "tenant_id": 123,
  "occurred_at": "2026-09-08T10:00:00+00:00",
  "payload": { "...": "..." }
}
```

Consumer এই envelope-টা জেনেরিকভাবে ডিকোড করে (`KafkaConsumer::decode()` — কোনো মেসেজ খারাপ হলে সেটা লগ করে স্কিপ করে, কিন্তু offset তবুও এগিয়ে যায়, যাতে একটা খারাপ মেসেজ পুরো topic-টাকে চিরদিনের জন্য আটকে না রাখে) আর handler-কে `{tenant_id, event_type, payload}` পাঠায়। টেন্যান্ট-নির্দিষ্ট কাজগুলো — `Org` খুঁজে বের করা, সেই টেন্যান্টের নিজের DB-তে `TenantManager` reconnect করা, তারপর লেখা — সবটাই handler-এর দায়িত্ব। **একটা consumer প্রসেস কিন্তু কোনো নির্দিষ্ট টেন্যান্টের না** — একটা `kafka:consume` প্রসেস তার হাতে থাকা যেকোনো partition-এর সব টেন্যান্টের ইভেন্ট হ্যান্ডেল করে, প্রতিটা flush-এর ভেতরে টেন্যান্ট-গ্রুপ অনুযায়ী DB কানেকশন বদলাতে বদলাতে।

---

৭. DevOps — মনোলিথের ভেতরেই একে মাইক্রোসার্ভিসের মতো চালানো

এখানকার Kafka consumer-গুলো queue worker না (Redis/DB queue টেবিলের সাথে কোনো সম্পর্ক নেই), আবার রিকোয়েস্টে চালিতও না — এগুলো **আলাদা, দীর্ঘস্থায়ী প্রসেস**, যেটা আপনি একটা ছোট, একক-উদ্দেশ্যের সার্ভিসের মতোই ডিপ্লয় আর সুপারভাইজ করেন — যদিও কোডটা `sokrio-server`-এর ভেতরেই থাকে।

৭.১ Supervisor — প্রোডাকশনে আসল প্রসেস মডেল

```
[program:sokrio-kafka-consumer-tracking]
command=php /var/www/html/artisan kafka:consume tracking
autostart=true
autorestart=true
numprocs=1
stopsignal=TERM
; stopwaitsecs অবশ্যই kafka.consumer.session_timeout (30s)-এর চেয়ে বেশি হতে হবে,
; নাহলে Supervisor SIGKILL করার আগে graceful SIGTERM-এর leaveGroup() করার সময় পাবে না।
stopwaitsecs=35
redirect_stderr=true
environment=DD_PROFILING_ENABLED="false",DD_TRACE_CLI_ENABLED="false"
stdout_logfile=/var/www/html/storage/logs/kafka-consumer-tracking.log
```

নতুন কোনো topic-এর consumer বসানোর সময় খেয়াল রাখার মতো জিনিস:

- **প্রতি topic-এ একটা প্রোগ্রাম ব্লক**, `command=php artisan kafka:consume <topic-key>`।
- `stopwaitsecs` অবশ্যই `KAFKA_CONSUMER_SESSION_TIMEOUT`-এর (ডিফল্ট 30s) চেয়ে বেশি রাখতে হবে — SIGKILL হওয়ার আগে SIGTERM-কে পরিষ্কারভাবে `leaveGroup()` করার সময় দিতে হয়, না দিলে প্রতিটা ডিপ্লয়ে একটা অগোছালো rebalance হয়ে যাবে।
- `environment=DD_PROFILING_ENABLED="false",DD_TRACE_CLI_ENABLED="false"` — এটা **শুধু** এই প্রোগ্রামেই দিতে হবে, পুরো container-এ না (§৪.৩)। ভুল করলে হয় আগের EINTR সমস্যা আবার ফিরে আসবে (চালু রাখলে), নাহলে অন্য জায়গায় চুপচাপ tracing হারিয়ে যাবে (সব জায়গায় বন্ধ করলে)।
- আপাতত `numprocs=1`। সত্যিকারের horizontal scale-out করতে এটা 1-এর বেশি করা অর্থবহ (একই group-এ বেশি partition parallel-এ consume করা যাবে), কিন্তু এই কোডবেসে এটা এখনও কখনো পরীক্ষা করা হয়নি — করলে শুধু "চালু হয়ে গেছে" দেখেই খুশি হয়ে যাবেন না, লোডের নিচে rebalance কেমন আচরণ করছে সেটা যাচাই করার সময়টা বাজেটে রাখুন।

৭.২ যা লাগবেই: `ext-pcntl`

সিগন্যাল হ্যান্ডলিং (`pcntl_signal`, `pcntl_async_signals`, `pcntl_sigprocmask`) করতে `pcntl` PHP এক্সটেনশন লাগে, যেটা php-fpm/mod_php-এ **পাওয়াই যায় না** — শুধু CLI SAPI-তে থাকে। `composer.json`/`composer.lock`-এ `ext-pcntl` require করা আছে। `kafka:consume` চালাবে এমন যেকোনো container image-এ (আর platform-req বাম্পের কারণে, `--ignore-platform-reqs` ছাড়া `composer install` চালানো যেকোনো environment-এই) এটা ইনস্টল থাকতেই হবে — নতুন কোনো deploy target বসানোর সময় এটা আগে থেকেই চেক করে নিন, কারণ এটা মিসিং থাকলে রানটাইমে গিয়ে টের পাবেন, তার আগে না।

৭.৩ কনফিগ — `config/kafka.php` / `.env.example`

সব Kafka টিউনিং env দিয়ে কন্ট্রোল করা হয়, আর producer আর consumer-এর জন্য আলাদা আলাদা — কারণ দুটোর লেটেন্সি বাজেট একদম উলটো: producer রিকোয়েস্ট পাথে থাকে বলে দ্রুত ফেল করতে হয়, আর consumer ব্যাকগ্রাউন্ড

প্রসেস বলে একটা সুস্থ round-trip-এর জন্য বেশি সময় অপেক্ষা করতে পারে।

Var	ডিফল্ট	কী করে
`KAFKA_BROKERS`	`localhost:9092`	Bootstrap সার্ভার
`KAFKA_GROUP`	`sokrio-group`	Consumer group id — partition ownership যাদের মধ্যে ভাগাভাগি হওয়ার কথা, সব consumer-এর group একই থাকতে হয়
`KAFKA_PRODUCER_CLIENT_ID` / `KAFKA_CONSUMER_CLIENT_ID`	`sokrio-producer` / `sokrio-consumer`	আলাদা রাখা হয়েছে যাতে broker-সাইড মেট্রিক্স/quota দুই role আলাদাভাবে দেখা যায়
`KAFKA_PRODUCER_*_TIMEOUT`	প্রতিটাই 1.0s	দ্রুত ফেল করতে হয় — এটা একটা HTTP রিকোয়েস্টের মধ্যেই চলছে
`KAFKA_PRODUCE_RETRY` / `KAFKA_MAX_WRITE_ATTEMPTS`	0 / 1 (লাইব্রেরি লেভেলে) — এর উপরে `KafkaProducer` নিজে 3 বার পর্যন্ত রিট্রাই করে (`MAX_PUBLISH_ATTEMPTS`)	Publish-এর রিলায়েবিলিটি
`KAFKA_CONSUMER_*_TIMEOUT`	3-5s	বেশি সময় দেওয়া — coordinator-এর round-trip একটু ধীর হলেই যেন dead socket মনে না হয়
`KAFKA_POLL_INTERVAL`	1.0s	খালি poll-এর মাঝে বিরতি (একটানা spin না করার জন্য)
`KAFKA_CONSUMER_SESSION_TIMEOUT` / `_REBALANCE_TIMEOUT` / `_HEARTBEAT`	30s / 30s / 3s	Group coordination প্রোটোকলের টাইমিং
`KAFKA_CONSUMER_MAX_CONSECUTIVE_FAILURES`	10	পরপর এতগুলো ফেইলিওরের পর (মাঝে একটাও সাফল্য না পেলে) Supervisor রিস্টার্টের জন্য exit করে
`KAFKA_CONSUMER_RECONNECT_BACKOFF_CAP`	30s	reconnect চেষ্টার মাঝের অপেক্ষার সর্বোচ্চ সীমা

Var	ডিফল্ট	কী করে
`KAFKA_CONSUMER_BATCH_SIZE` / `_BATCH_INTERVAL`	200 / 1.0s	Flush হবে কখন — যেটা আগে হয়
`KAFKA_CONSUMER_MAX_RUNTIME`	21600 (6h)	পিরিয়ডিক clean-exit রিসাইকেল, `0` দিলে বন্ধ
`KAFKA_AUTO_CREATE_PARTITIONS` / `_REPLICATION_FACTOR`	3 / 1	`TopicProvisioner` মিসিং topic অটো বানানোর সময় ইউজ করে (dev/staging-এর জন্যই — production-এ এর উপর নির্ভর না করে ops টিম যে partition/replication ভ্যালু ঠিক করেছে সেভাবে আগে থেকেই topic বানিয়ে রাখুন)
`KAFKA_REPORT_UNEXPECTED_PUBLISH_ERRORS`	false	চালু করলে `kafka` লগ চ্যানেলের পাশাপাশি অপ্রত্যাশিত publish ফেইলিওরও Sentry-তে (`report()`) যাবে

৭.৪ লোকালি একটা consumer চালিয়ে দেখা

```
# container-এর ভেতরে / ./dev pa দিয়ে
php artisan kafka:consume tracking

# আরেকটা টার্মিনালে, tracking endpoint হিট করে একটা publish ট্রিগার করুন
# (TrackingController::store ইতিমধ্যেই producer-কে কল করে দেয়)
curl -X POST https://<tenant>.sokrio-server.test/api/trackings ...

# লগে দেখুন ইভেন্ট পৌঁছালো কিনা
tail -f storage/logs/kafka.log
```

লোকালি একদম নতুন একটা topic end-to-end বানাচ্ছেন? `TopicProvisioner` প্রথম publish-এই topic বানিয়ে দেয় (এটা শুধু dev-এর সুবিধা, staging/prod-এ এর ভরসায় থাকবেন না — উপরে বলা আছে), তাই লোকাল ইটারেশনের জন্য ম্যানুয়ালি `kafka-topics --create` করার দরকার নেই।

৭.৫ মনিটরিং — "ঠিকঠাক চলছে" মানে কী

- `storage/logs/kafka.log` (+ stderr) — Kafka-র সব লগ এখানেই যায়। প্রায় 6h পরপর `Kafka consumer: clean shutdown` মানে পিরিয়ডিক রিসাইকেল, স্বাভাবিক। ঘনঘন `mid-poll error, reconnecting` দেখলে broker/নেটওয়ার্কের স্বাস্থ্য চেক করুন। আর `repeated mid-poll failures,`

`exiting for supervisor restart` দেখলে ধরে নিন — এটা একটা আসল ইনসিডেন্ট, Supervisor রিস্টার্ট করে দেবে ঠিকই, কিন্তু upstream-এ কোথাও একটা সমস্যা আছে।

- ডিফল্টভাবে এখান থেকে কিছুই Sentry/Slack-এ যায় না (§৪.৬, §৫.৩)। Kafka-র স্বাস্থ্যের উপর alert দরকার হলে সেটা নিজে থেকে বানাতে হবে — `kafka` চ্যানেলের উপর log-based alert বসিয়ে, অথবা `KAFKA_REPORT_UNEXPECTED_PUBLISH_ERRORS` চালু করে। এমনি এমনি হবে না।

---

৮. চেকলিস্ট — নতুন topic + consumer যোগ করার সময়

পরের কোনো pipeline বানানোর সময় এটা ধরে এগোন (এই ফোল্ডারেরই Primary Order/Delivery প্ল্যানটা পরবর্তী real উদাহরণ — সেটার সাথে মিলিয়ে দেখুন):

```
[ ] TopicRegistry::TOPICS-এ topic key + name, event-type কনস্ট্যান্ট যোগ করুন
[ ] Producer সাইড: ইভেন্টটা যেখান থেকে আসলেই উৎপন্ন হয় সেখান থেকে publish করুন
    - একটা রিকোয়েস্ট-টাইম Controller (TrackingController-এর মতো), অথবা
    - PublishesKafkaMessage trait দিয়ে একটা model Observer (Order/
      DeliveryObserver-এর মতো — কিন্তু publish কলটা ENABLED রাখুন, retail
      পেয়ারের মতো এখনকার কমেন্ট-আউট করে রাখবেন না)
    - thin (শুধু id পাঠানো, consumer বাকিটা নিজে বের করে নেয়) না fat (পুরো
      payload একসাথে পাঠানো) — কোনটা নেবেন সেটা ঠিক করুন, ট্রেড-অফটা Primary
      Order/Delivery প্ল্যানের §৩-এ আছে
    - বিশেষ কোনো কারণ না থাকলে partition key = tenant_id
[ ] Consumer সাইড: KafkaMessageHandlerInterface ইমপ্লিমেন্ট করা একটা নতুন ক্লাস
    বানান, শুধু handleBatch() লিখুন (handle() deprecated) — tenant_id দিয়ে
    গ্রুপ করুন, প্রতি টেন্যান্ট প্রতি flush-এ একটা TenantManager reconnect +
    একটা bulk write, সীমিত রিট্রাই, handleBatch থেকে কখনো exception বের হতে
    দেবেন না
[ ] নতুন handler-টা ConsumeCommand::HANDLERS-এ রেজিস্টার করুন
[ ] নতুন Supervisor প্রোগ্রাম ব্লক বানান (§৭.১), stopwaitsecs যেন
    session_timeout-এর চেয়ে বেশি হয়, DD_PROFILING_ENABLED=false /
    DD_TRACE_CLI_ENABLED=false সেই প্রোগ্রামে দিন
[ ] এই consumer যে box/image-এ চলবে সেখানে ext-pcntl আছে কিনা কনফার্ম করুন
[ ] টেস্ট লিখুন: handler-এর batch behavior (অজানা টেন্যান্ট, সফল হওয়া,
    retry-then-fail), TopicRegistry key কভারেজ —
ConsumeCommandHandlersTest-এর
    reflection guard রেজিস্টার হওয়ার পর নিজে থেকেই interface conformance
    কভার করে ফেলবে
[ ] পুরনো কোনো sync/batch write path রিপ্লেস করছেন? Kafka পাথ ভ্যালিডেট না হওয়া
    পর্যন্ত পুরনো পাথটা প্যারালালে চালু রাখুন — target schema নতুন হলে শ্যাডো
    কালেকশন ব্যবহার করুন, একই কালেকশন হলে দুই আউটপুট মিলছে কিনা দেখেই বুঝুন
    (§৪.২ দেখুন)
```

---

৯. শব্দকোষ — Kafka একদম নতুন যাদের কাছে

- **Broker** — একটা Kafka সার্ভার প্রসেস; `KAFKA_BROKERS`-এ এক বা একাধিক broker-এর ঠিকানা থাকে।
- **Topic** — মেসেজের একটা নাম-দেওয়া, পার্টিশনে ভাগ করা লগ।
- **Partition** — একটা topic-এর ভেতরে একটা অর্ডারড, অপরিবর্তনীয় sub-log। অর্ডার গ্যারান্টি শুধু একটা partition-এর ভেতরেই কাজ করে।
- **Offset** — একটা partition-এর নিজস্ব sequence number, মানে "এই consumer group কতদূর পড়েছে"।

- **Consumer group** — partition ownership ভাগাভাগি করা consumer-দের একটা নাম-দেওয়া দল; কোনো member join/leave করলে broker নিজে থেকে partition assignment rebalance করে দেয়।
- **Rebalance** — group membership বদলালে (কেউ join/leave/restart করলে) চলা প্রোটোকল প্রক্রিয়া — partition নতুন করে assign হয়, ততক্ষণে ওই partition-গুলোর consumption সংক্ষিপ্ত সময়ের জন্য থেমে থাকে।
- **At-most-once / at-least-once / exactly-once** — read আর processed-এর মাঝে কোথাও ফেইল করলে কী হবে, সেটা বোঝানোর delivery গ্যারান্টি। রিট্রাই শেষ করে ফেলা একটা handler-এর জন্য আমরা কার্যত **at-most-once** (§২-র Delivery semantics সারিতে বলা আছে) — মনে রাখবেন, Kafka নিজে থেকে কোনো durability গ্যারান্টি দেয় না, সেটা handler নিজে অর্জন করে নিতে হয়।
- **EINTR** — কোনো সিগন্যাল একটা blocking syscall-কে শেষ হওয়ার আগে interrupt করলে যে errno রিটার্ন হয়; Datadog profiler-এর কারণে pure-PHP Kafka ক্লায়েন্টে এটা কেন বিশেষভাবে গুরুত্বপূর্ণ, সেটার জন্য §৪.৩ দেখুন।
- **KIP-345** — Kafka Improvement Proposal যেটা static consumer group membership যোগ করেছিলো (ছোট রিস্টার্টে পুরো rebalance এড়াতে একটা ফিক্সড `group.instance.id`) — আমরা কেন এর বদলে *dynamic* (non-static) membership চালাচ্ছি, সেটার জন্য §৪.৪ দেখুন।

---

১০. এরপর কী পড়বেন

- **পরের real বিল্ড:** এই একই ফোল্ডারের `primary-order-delivery-kafka-plan.md` — এই ডকের সবকিছু একটা real, এখনও-ব্যাচ pipeline-এ (Primary Order/Delivery সামারি) কীভাবে প্রয়োগ হবে সেটা দেখায়, "thin না fat payload" সিদ্ধান্তটা সহ, আর কেন বিদ্যমান `SummariesQuery` লজিক নতুন করে না বানিয়ে পুনর্ব্যবহার করাই ভালো সেটাও।
- **PR #5094 ব্রেকডাউন:** `_sprintSep/Kafka/order_summary/PR-5094-breakdown.md` — Tracking-pilot যুগের কোডের Aug 2026-এর একটা স্ন্যাপশট। এটাকে ইতিহাস হিসেবে পড়ুন, বর্তমান অবস্থা হিসেবে না — এই ডকের §৪.৭/§৪.৮/§৭-এ ওই PR-এর পরে যা যা বদলেছে (batch processing, dynamic group membership, 6h max runtime) সেসব আছে।
- **আসল সোর্স অফ ট্রুথ:** §৩-এর ফাইল ম্যাপ — সন্দেহ হলে মনে রাখবেন, এই ডকসহ যেকোনো ডকের চেয়ে কোডটাই বেশি আপ-টু-ডেট ও সঠিক।